

AUTOMATED VOICE GROUP DISCUSSION (GD) SYSTEM

Using Self-Hosted Jitsi + Rule-Driven Orchestration

1. PURPOSE AND SCOPE

This document defines a **fully automated, recruiter-less, AI-less Voice Group Discussion (GD) system** designed for job portals and assessment platforms.

The system converts a traditionally subjective, moderator-driven GD into a **deterministic protocol** where:

- Speaking is enforced, not requested
- Order is computed, not negotiated
- Evaluation is numeric, not interpretive
- Bias is structurally eliminated through protocol design

Human judgment is removed.

AI judgment is removed.

Only **rules, enforcement, and logs** remain.

2. CORE DESIGN PHILOSOPHY

Replace judgment with protocol

Replace observation with instrumentation

Replace moderation with state machines

The system explicitly avoids:

- Confidence scoring
- Personality traits
- Leadership labels

It operates only on:

- Permission

- Time
- Compliance
- Participation

The system does not attempt to “understand” candidates.
It **measures behavior under enforced constraints**.

3. HIGH-LEVEL ARCHITECTURE

Separation of Responsibilities

Layer	Responsibility
Your Portal	Brain (rules, flow control, scoring)
Jitsi	Voice transport only
WebRTC	Audio communication protocol
Backend Orchestrator	Enforcement and state control

Critical principle

Jitsi does not decide anything.

WebRTC does not decide anything.

Your backend decides everything.

4. WHY JITSI (TECHNICAL RATIONALE)

What Jitsi Is

- Open-source WebRTC-based conferencing stack
- Browser-native (no installs required)
- Fully self-hostable
- No per-minute or per-user cost
- Uses SFU architecture for stability and scale

Core Jitsi Components

- **Jitsi Meet** — user-facing meeting layer
- **Jitsi Videobridge** — SFU that routes audio streams
- **Jicofo** — conference focus and participant coordination

Why It Fits Voice GD

Requirement	Supported
Voice-only discussion	Yes
No recruiter present	Yes
Automated room lifecycle	Yes
Forced mute / unmute	Yes
Turn-based speaking	Yes (via API control)
Free / unpaid	Yes (self-hosted)
Metadata capture	Yes
Audio recording	Intentionally avoided

Jitsi functions as a **controlled microphone layer** and nothing more.

5. WHY NOT ZOOM / GOOGLE MEET

Feature	Jitsi	Zoom	Google Meet
Cost	Free	Paid	Paid
Login required	No	Yes	Yes
Open source	Yes	No	No
Self-hosting	Yes	No	No
Automation control	Full	Limited	Limited

Zoom and Google Meet are **end-user products**.
Jitsi is **programmable infrastructure**.

6. SYSTEM COMPONENTS (LOGICAL)

6.1 Communication Layer

- WebRTC for real-time audio
- Jitsi SFU for audio routing
- No business logic

6.2 Session Orchestrator (CRITICAL)

Backend service responsible for:

- Room creation
- Participant ordering
- Speaking windows
- Forced mute / unmute
- Session termination
- Failure handling

Implementation stack:

- Node.js or Spring Boot
- Redis (deterministic state machine)
- WebSockets (real-time command channel)

7. DETERMINISTIC GD FLOW (END-TO-END)

7.1 Pre-GD Phase (Automatic)

- Topic assigned from database
- Candidates mapped into batch
- Schedule fixed
- Room name generated
- No approvals
- No host

Example:

gd_banking_20240206_1234

7.2 Entry Lock

- Join window strictly enforced
- Late joiners become spectators only
- No exceptions

This removes tactical lateness.

8. VOICE GD EXECUTION FLOW

8.1 Frontend (Before Voice)

- Topic displayed
- Preparation timer (10–20 minutes)
- Rules shown
- Jitsi **not** loaded

This prevents bandwidth waste and premature microphone access.

8.2 Jitsi Room Creation (Backend)

- Room created programmatically
- No manual links
- No sharing

8.3 Joining the Room (Audio-Only)

- Video disabled
- Audio muted by default
- UI controls removed

Jitsi remains invisible infrastructure.

9. TURN-BASED SPEAKING ENGINE

This replaces the human moderator entirely.

Conceptual Algorithm

```
participants = sort_by_join_time()
```

```
for each participant:
```

```
    grant_speaking_token(duration = X seconds)
```

```
    force_unmute(active_user)
```

```
    force_mute(all_others)
```

Enforcement Rules

- Only the token holder's mic is open
- Countdown timer visible
- Timer expiry triggers auto-mute
- No overlap possible

- Interrupt attempts logged

Examples:

- Intro round: 60 seconds
- Rebuttal round: 30 seconds
- Conclusion round: 45 seconds

All actions enforced by backend state machine.

10. OPEN DISCUSSION ROUND

After structured rounds:

- All participants unmuted
- Speaking time still tracked
- Interruptions still logged
- Dominance penalized numerically

Freedom exists only under measurement.

11. DATA CAPTURE (NO AUDIO STORAGE)

Captured

- Mic open duration (seconds)
- Turns completed
- Turns skipped
- Interrupt attempts
- Silence during token
- Network drops
- Early exits

Not Captured

- Voice recordings
- Speech content
- Tone or emotion
- Accent

This guarantees privacy and zero storage cost.

12. SCORING (RULE-BASED)

Example:

```
score =  
+ (used_all_turns ? 10 : 0)  
+ (spoke_full_time ? 10 : 0)  
- (interrupt_attempts * 2)
```

No confidence metrics.

No leadership inference.

Only compliance and participation.

Scores are reproducible and auditable.

13. FAILURE HANDLING

Failure	System Action
Network drop	Token skipped
Mic unused	Logged as silence
User exits	Removed from queue
Rejoin attempt	Denied

No retries.

No discretionary logic.

14. BANDWIDTH CHARACTERISTICS (VOICE-ONLY)

Typical per user:

- Upload: 40–60 kbps
- Download: 150–250 kbps (5–6 users)

Stability reasons:

- SFU-based routing
 - Server-mediated streams
 - Mobile-network tolerant
-

15. DEPLOYMENT (ZERO-COST STACK)

Hosting

- Oracle Cloud Free Tier (preferred)
- AWS Free Tier (limited)
- Local VM (MVP)

Stack

- Media: WebRTC + Jitsi
- Signaling: WebSocket
- State: Redis
- TURN: coturn
- Hosting: Free VM

No paid services.

16. ROLE IN A JOB PORTAL

Use cases:

- Recruiter-less GD rounds
- Campus hiring
- Mass screening
- Communication discipline training

Your portal controls:

- Entry
- Order
- Time
- Scoring

Jitsi only transmits sound.

17. SYSTEM DEFENSE

“This system removes humans and AI entirely by converting group discussion into a deterministic protocol where behavior is enforced, logged, and evaluated purely through rules.”

18. WHAT IS WEBRTC

WebRTC (Web Real-Time Communication) is a browser-native standard enabling real-time audio, video, and data transfer without plugins.

In this system, WebRTC:

- Captures microphone audio
- Encrypts streams (SRTP)
- Handles echo cancellation and noise suppression

WebRTC does **not**:

- Decide turn order
- Enforce rules
- Score participants

WebRTC is the transport layer only.

19. HOW WEBRTC OPERATES (VOICE-ONLY)

1. Mic permission
2. Audio stream creation
3. Signaling (SDP / ICE exchange)
4. NAT traversal via STUN/TURN
5. Encrypted audio flow
6. Programmatic mute/unmute
7. Connection teardown

The backend never accesses raw audio.

20. WHAT IS JITSI

- WebRTC-based conferencing platform
- Open source
- Self-hosted
- Programmable

WebRTC is the protocol.
Jitsi is the implementation.

21. WHAT JITSI PROVIDES / DOES NOT PROVIDE

Provides:

- Complete WebRTC stack
- SFU routing
- IFrame + JS API
- Zero license cost

Does not provide:

- GD rules
- Fairness logic
- Evaluation

Jitsi obeys commands. It does not judge.

22. INTEGRATION MODEL

Candidate Browser



Your App UI



Jitsi IFrame (audio-only)



Self-Hosted Jitsi Server

23. IMPLEMENTATION SUMMARY

- Jitsi hosted once
- Rooms created programmatically
- Frontend embeds restricted Jitsi
- Backend controls mic state and time

Jitsi executes. Backend governs.

24. WHY JITSI IS FREE AND OPEN SOURCE

- Apache 2.0 license
- No usage fees
- No vendor lock-in
- Auditable source

Cost = infrastructure only.

25. ALTERNATIVES

Open-Source

- LiveKit
- Mediasoup
- Janus

Paid SaaS

- Agora
- Twilio Video
- Daily.co
- Vonage

Paid tools optimize convenience.
This system optimizes control and determinism.

26. FINAL CLARITY

- WebRTC is the physics
- Jitsi is the engine
- Backend is the law
- Candidates operate under constraint
- Output is measurable behavior